

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

Submmited by	
Name	**CH.SATHWIK**
Roll No	**AV>SC>U4CSE25025**
Year/sem/section	**2nd year/sem-3/CSE-A**
Credits	**5**

Submmited to	
Name	**DR.RAJKUMAR BATCHU**
Department	**School of computing**
Designation	**Asst.professor(SI.Grade)**

INDEX

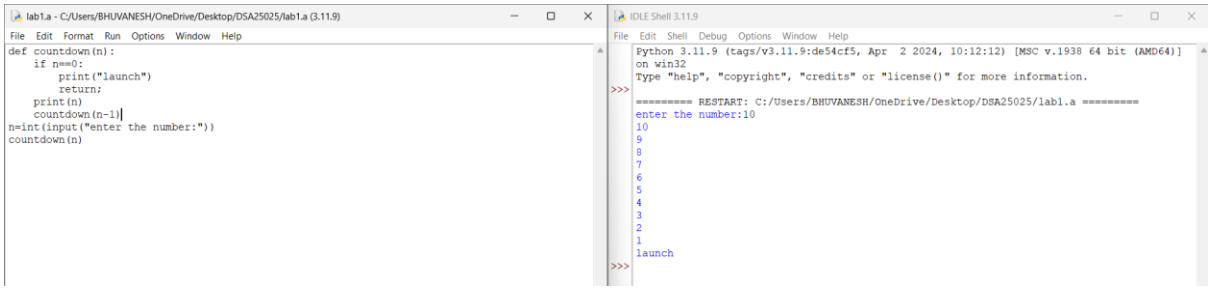
s.no	title	Date
1	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	23-07-26
2	WEEK 2: 2a. Write a python program to perform linear search on the give input. 2b. Write a python program to perform binary search on the given. Verify by taking sorted array as an input. 2c. Write a python program to perform binary search on the given. Verify by taking unsorted array as an input.	30-07-26
3	WEEK-3: 3a.write a python program to perform bubblesort on the given input 3b. write a python program to perform insertion_sort on the given input 3c. write a python program to perform selection sort by taking given input 3d.write a python program to perform merge sort by taking the given input . 3e. write a python program to perform quick sort by taking given input	06-08-26
4	Week-4:4a.write a python program to perform linked list operations by taking the input from the user 4b. write a python program to perform double linked list operations by taking input from the user 4c. write a python program to perform circular linked list operations by taking input from the user	13-08-26
5	Week-5: 5a write a python program to implement the stack by using arrays. 5b. write a python program to implement stack using the linked list	27-08-26
6	Week-6: 6a write a python program to implement the queue by suing the arrays. 6b, write a python program to implement the queue by using the linked list.6c. write a python progam to implement the circular queue using the arrays	3-09-26
7	Week-7 :7a write a python program to implement the binary tree using arrays 7b: write a python program to implement the binary tree using linked list	24-09-26
8	Week-8 8a write a python program to implement the binary search tree using the arrays	8-10-26
9	Week-9 write a python program to implement the AVL tree using arrays	22-10-26

10	Week-10 write a python program on hashing separate chaining , linear , quadratic probing	29-10-26

WEEK-1

1A).AIM:A Rocket launch system displays a countdown before launch. Instead of suing a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1 followed by “launch”.

Code:



Explanation

Condition 1: Positive Input (n > 0)

- The function checks whether n == 0.
- Since n is positive, the condition is **False**.
- It executes:
  - print(n) → prints the current value of n.
  - countdown(n-1) → calls the same function again with n-1.
- This process continues until n becomes 0.
- When n == 0, the condition becomes **True**.
- It prints **launch**.
- The return statement stops that function call.
- Then the recursive calls return one by one.
- Since there is no value returned by the function, it returns **None**.

Condition 2: Zero Input (n == 0)

- The function checks if n == 0.
- The condition is immediately **True**.
- It prints **launch**.

- The return statement stops the function.
- No recursive call takes place.
- The function returns **None**.

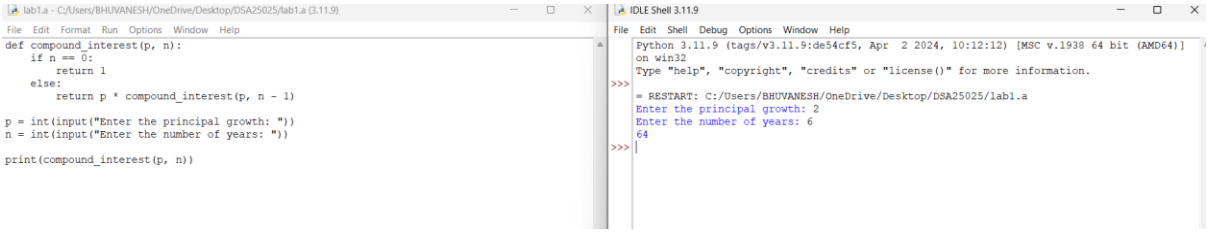
Condition 3: Negative Input (n < 0)

- The condition n == 0 is **False**.
- It prints the negative value.
- Then it calls itself with n-1, making the value even more negative.
- Therefore, it will **never reach 0**.
- The recursion continues indefinitely.
- Eventually, Python raises a **RecursionError** because the maximum recursion depth is exceeded

S.no	ERROR OBTAINED	RECTIFICTION
1	Negative input in countdown	Add a condition to handle negative numbers before recursion

1B). Aim: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p(n) where p=principal growth, n= no. of years.

Code:



Explanation

Condition 1: n > 0

- The function compound_interest(p, n) is called with the principal amount p and number of years n.
- The function checks whether n == 0.
- Since n is greater than 0, the condition is **False**.
- It executes the else block.
- It calculates:

- p * compound_interest(p, n-1)
- The function calls itself recursively with n-1.
- This process continues until n becomes 0.
- Each recursive call multiplies p one more time.
- Therefore, the function calculates $\mathbf{p^n}$.

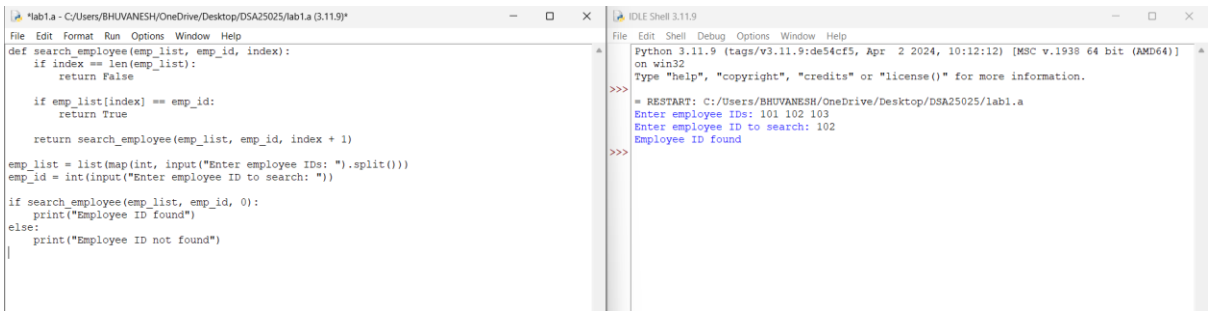
Condition 2: n == 0

- When n becomes 0, the condition n == 0 becomes **True**.
- The function returns 1.
- This 1 acts as the **base case** for the multiplication.
- The recursive calls then return one by one.
- Each returned value is multiplied by p.
- Finally, the result is printed using print(compound_interest(p,n))

S.no	ERROR OBTAINED	RECTIFICTION
1	**Wrong indentation**	**Maintain proper indentation**

1C). AIM . An Hr department stores employee IDs in a list .Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID in a list.

Code and output:



Explanation

Condition 1: index == len(emp_list)

- The function checks whether index has reached the length of the employee list.

- If index == len(emp_list), it means we have checked **all employee IDs**.
- The employee ID was not found.
- The function returns **False**.
- No further recursion takes place.

Condition 2: emp_list[index] == emp_id

- The function checks whether the current employee ID is equal to the ID we are searching for.
- If both IDs are equal, the condition becomes **True**.
- The function returns **True**.
- The recursion stops because the employee ID has been found.

Condition 3: Employee ID does not match

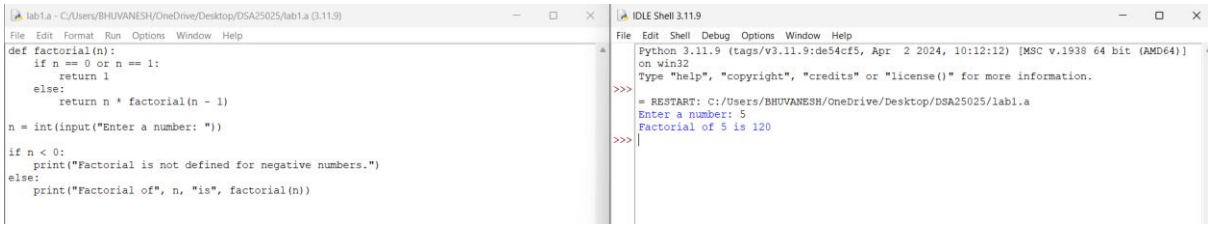
- If the current employee ID is not equal to emp_id, the condition is **False**.
- The function calls itself recursively with:
  - index + 1
- This moves to the **next employee ID** in the list.
- The process continues until:
  - The employee ID is found, or
  - The entire list is checked.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	**Missing return statement**	**Add the required return statement**

1D. AIM. In a logistics company , the no . of possible ways to arrange in different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code and output:



Explanation

Condition 1: Negative Input (n < 0)

- The program first checks whether n < 0.
- If n is negative, the condition is **True**.
- Factorial is not defined for negative numbers.
- It prints:
  Factorial is not defined for negative numbers.
- The factorial() function is **not called**.

Condition 2: Base Case (n == 0 or n == 1)

- Inside the factorial() function, it checks:
  n == 0 or n == 1
- If n is 0 or 1, the condition is **True**.
- The function returns **1**.
- This is the **base case** of the recursion.
- It stops further recursive calls.

Condition 3: n > 1

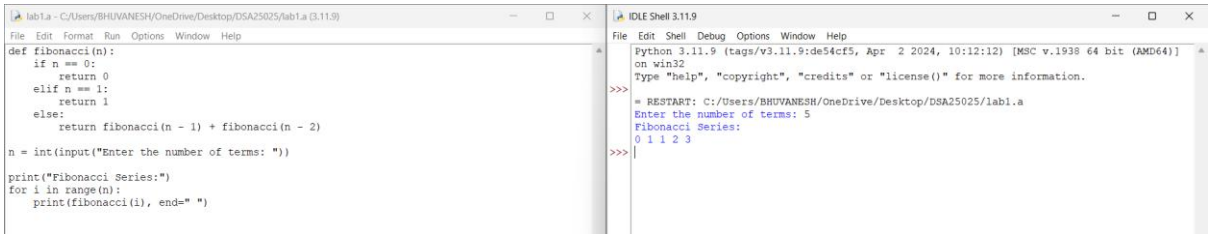
- If n is greater than 1, the condition n == 0 or n == 1 is **False**.
- The else block is executed.
- It returns:
  n * factorial(n - 1)
- The function calls itself with n - 1.
- This continues until n becomes 1.
- Then the returned values are multiplied together

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	**Wrong recursive value**	**Updated the recursive value correctly**

1E). AIM: Write a recursive program to print the first n terms of the fibonacci series

Code and output:



```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter the number of terms: "))

print("Fibonacci Series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab1.a
Enter the number of terms: 5
Fibonacci Series:
0 1 1 2 3
>>>
```

Explanation

Condition 1: n == 0

- The function checks whether n == 0.
- If the condition is **True**, the function returns 0.
- This is the **first base case**.
- No further recursive call takes place.

Condition 2: n == 1

- If n is not 0, the function checks whether n == 1.
- If the condition is **True**, the function returns 1.
- This is the **second base case**.
- No further recursive call takes place

Condition 3: n > 1

- If n is neither 0 nor 1, both conditions are **False**.
- The else block is executed.
- The function calls itself **two times**:
  - fibonacci(n - 1)
  - fibonacci(n - 2)
- The two returned values are added together.
- This process continues until the base cases n == 0 and n == 1 are reached.

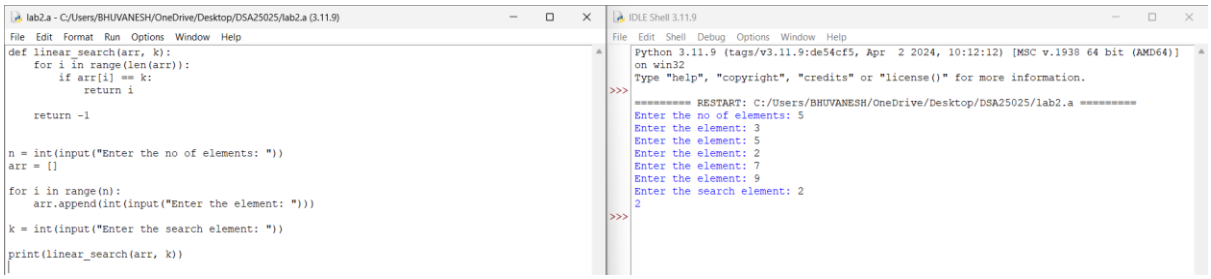
Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	**Incorrect Fibonacci recursion**	**Use fibonacci(n-1) + fibonacci(n-2).**

WEEK-2

2a. AIM: Write a python program to perform linear search on the give input.

Code and Output:



```
lab2.a - C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab2.a (3.11.9)
File  Edit  Format  Run  Options  Window  Help
def linear_search(arr, k):
    for i in range(len(arr)):
        if arr[i] == k:
            return i

    return -1


n = int(input("Enter the no of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter the element: ")))

k = int(input("Enter the search element: "))

print(linear_search(arr, k))
```

```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab2.a ==========
Enter the no of elements: 5
Enter the element: 3
Enter the element: 5
Enter the element: 2
Enter the element: 7
Enter the element: 9
Enter the search element: 2
2
>>>
```

Explanation :

Condition 1: Element is Found (arr[i] == k)

- The function linear_search(arr, k) receives the array arr and the search element k.
- The for loop starts from the first element of the array.
- It checks each element one by one using:
  arr[i] == k
- If the current element is equal to k, the condition becomes **True**.
- The function immediately returns the **index i**.
- Since the function returns the index, the loop stops immediately.
- The remaining elements are not checked.

Condition 2: Element is Not Found

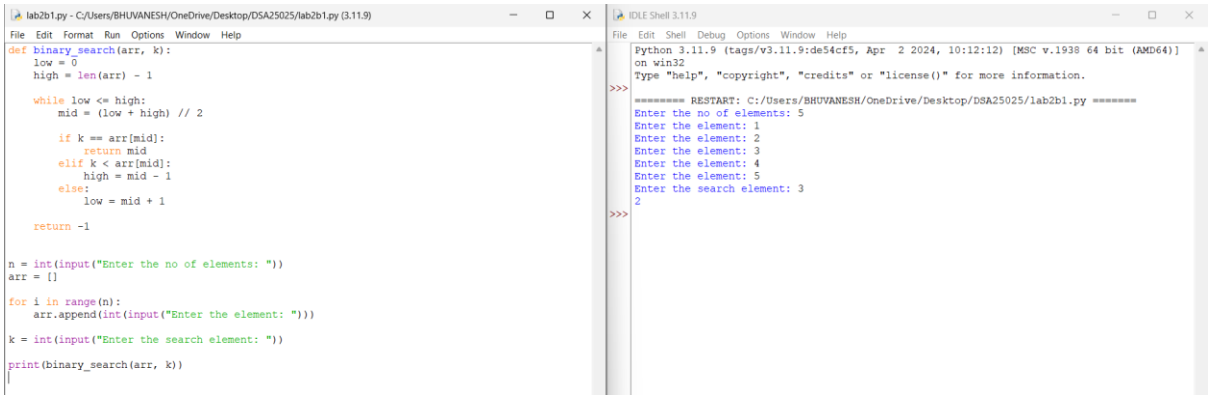
- The loop checks every element in the array.
- If none of the elements is equal to k, the if condition is always **False**.
- After the loop finishes, the statement return -1 is executed.
- -1 indicates that the search element is **not present** in the array.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	**Wrong loop range**	**Use range(n) to process exactly n elements.**

2b. AIM: Write a python program to perform binary search on the given. Verify by taking sorted array as an input.

Code and Output:



```
lab2b1.py - C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab2b1.py (3.11.9)
File  Edit  Format  Run  Options  Window  Help
def binary_search(arr, k):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if k == arr[mid]:
            return mid
        elif k < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1

    return -1


n = int(input("Enter the no of elements: "))
arr = []

for i in range(n):
    arr.append(int(input("Enter the element: ")))

k = int(input("Enter the search element: "))

print(binary_search(arr, k))
```

```
IDLE Shell 3.11.9
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>

======== RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab2b1.py ========
Enter the no of elements: 5
Enter the element: 1
Enter the element: 2
Enter the element: 3
Enter the element: 4
Enter the element: 5
Enter the search element: 3
2
>>>
```

Explanation

Condition 1: k == arr[mid] — Element Found

- The function binary_search(arr, k) receives the array and the search element.
- low is initialized to 0, which represents the first index.
- high is initialized to len(arr)-1, which represents the last index.
- The while loop runs as long as low <= high.
- mid is calculated using:
  mid = (low + high) // 2
- The middle element is compared with the search element k.
- If k == arr[mid], the element is found.
- The function returns the **index mid** and stops searching.

Condition 2: k < arr[mid] — Search Left Half

- If k is smaller than the middle element, the condition is **True**.
- This means the search element can only be present in the **left half**.
- Therefore:
  high = mid - 1
- The right half is eliminated.
- The process continues with the remaining left half

Condition 3: k > arr[mid] — Search Right Half

- If k is greater than the middle element, the elif condition is **False**.
- The else block is executed.

- This means the search element can only be present in the **right half**.
- Therefore:
  low = mid + 1
- The left half is eliminated.
- The process continues with the remaining right half.

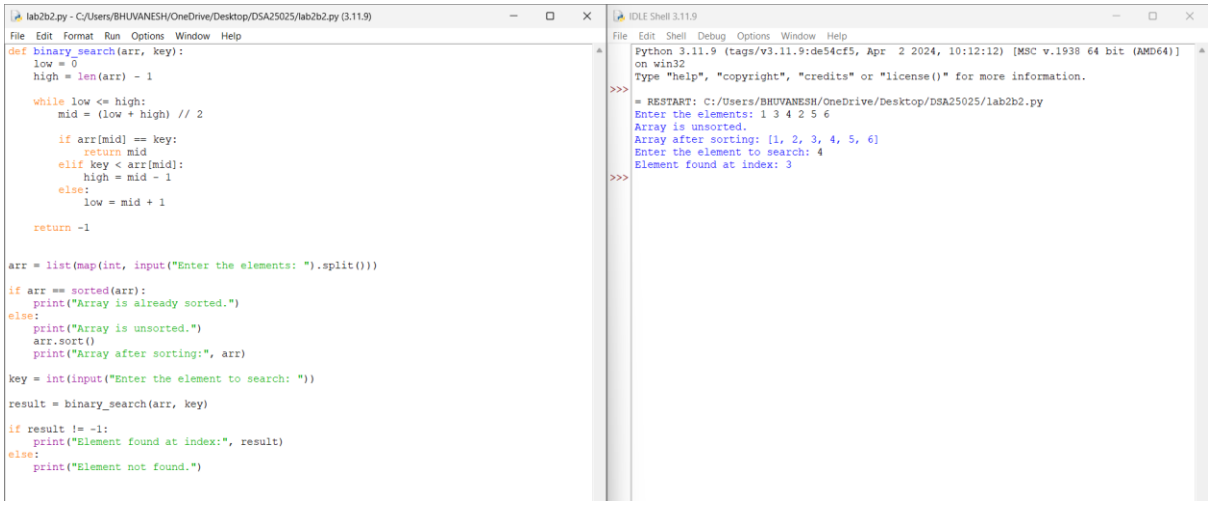
Condition 4: low > high — Element Not Found

- The while loop continues until low <= high becomes **False**.
- When low > high, there are no elements left to search.
- This means the search element is not present in the array.
- The function returns **-1**.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	**Wrong index in binary search**	**Use high = mid - 1 for the left half and low = mid + 1 for the right half.**

Output:



Explanation

Condition 1: arr[mid] == key — Element Found

- The function first calculates the middle index using mid = (low + high) // 2.

- It compares the middle element arr[mid] with the search element key.
- If arr[mid] == key, the condition is **True**.
- The element is found.
- The function returns the index mid.
- The search stops.

Condition 2: key < arr[mid] — Search Left Half

- If key is smaller than arr[mid], the condition is **True**.
- This means the required element may be present in the **left half**.
- The right half is ignored.
- high is changed to mid - 1.
- The search continues in the left half.

Condition 3: key > arr[mid] — Search Right Half

- If key is greater than arr[mid], the previous conditions are **False**.
- The element may be present in the **right half**.
- The left half is ignored.
- low is changed to mid + 1.
- The search continues in the right half.

Condition 4: low > high — Element Not Found

- The while low <= high loop continues until there are elements left to search.
- When low > high, the loop stops.
- This means the element is not present in the array.
- The function returns -1.
- The program prints **"Element not found."**

Condition 5: Array is Already Sorted

- The program checks whether the given array is already sorted.
- It uses:
  arr == sorted(arr)
- If the condition is **True**, it prints:
  "Array is already sorted."
- The array is directly used for binary search.

Condition 6: Array is Unsorted

- If arr == sorted(arr) is **False**, the array is unsorted.
- The program prints:
  "Array is unsorted."
- It then uses arr.sort() to sort the array.
- The sorted array is displayed.
- Binary search is then performed on the sorted array.

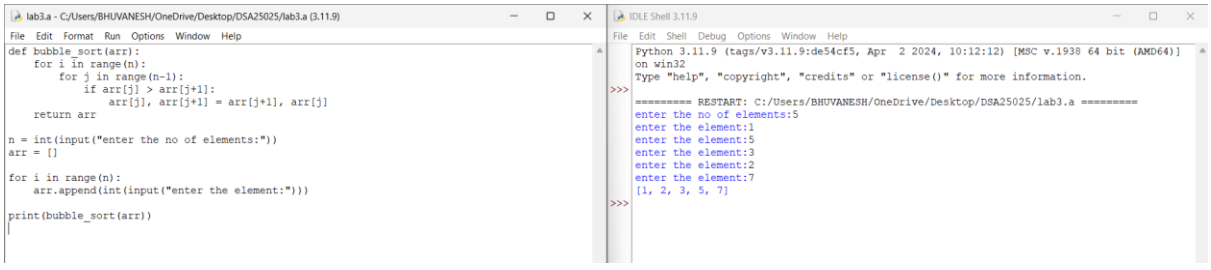
Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	**Binary search on unsorted array**	**sort the array first using arr.sort() or provide a sorted array.**

Week-3

3a.Aim.Write a python program to sort the elements using a bubble sort technique. Take an input elements from the user.

Code and Output:



Explanation

Condition 1: arr[j] > arr[j+1] — Elements Need to Be Swapped

- The function bubble_sort(arr) receives the array as an argument.
- The outer for loop runs from 0 to n-1 and controls the number of passes.
- The inner for loop runs from 0 to n-2 and compares adjacent elements.

- The condition arr[j] > arr[j+1] checks whether the current element is greater than the next element.
- If the condition is **True**, the two elements are in the wrong order.
- Therefore, the two elements are swapped using:
- Arr[j],arr[j+1]=arr[j+1],arr[j]
- After the swap, the smaller element moves toward the left and the larger element moves toward the right.

Condition 2: arr[j] <= arr[j+1] — No Swap Required

- If arr[j] > arr[j+1] is **False**, it means the current element is smaller than or equal to the next element.
- Therefore, the elements are already in the correct ascending order.
- No swapping is performed.
- The inner loop continues to compare the next pair of adjacent elements.

Condition 3: for i in range(n) — Repeated Passes

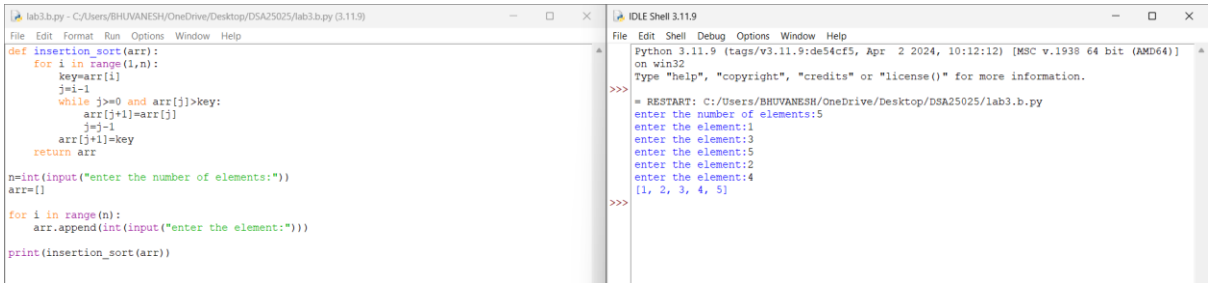
- The outer loop repeats the comparison process for all the elements.
- After each complete pass, the **largest unsorted element moves to the end of the array**.
- The process is repeated until all elements are arranged in ascending order.
- Finally, the function returns the sorted array using:

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	IndexError: list index out of range	Use correct loop range

3b.Aim. write a python program to sort the elements using a insertion sort technique. Take an input elements from the user.

Code and Output:



Explanation:

Condition 1: for i in range(1, len(arr)) — Selecting Element

- The loop starts from the **second element**.
- The first element is considered already sorted.
- i represents the element that needs to be placed correctly.

Condition 2: key = arr[i] — Storing Element

- The current element is stored in key.

```
key = arr[i]
```

- This element will be compared with the elements before it.

Condition 3: j = i - 1 — Getting Previous Position

- j points to the element just before the current element.

```
j = i - 1
```

Condition 4: while j >= 0 and key < arr[j] — Comparing Elements

- The current element is compared with the previous elements.
- If the previous element is greater than key, it is moved one position to the right.

```
while j >= 0 and key < arr[j]:
    arr[j + 1] = arr[j]
    j -= 1
```

- This creates space for the current element.

Condition 5: arr[j + 1] = key — Inserting Element

- The key is placed in its correct position.

```
arr[j + 1] = key
```

- The elements before the current position are now sorted.

Condition 6: return arr — Returning Sorted Array

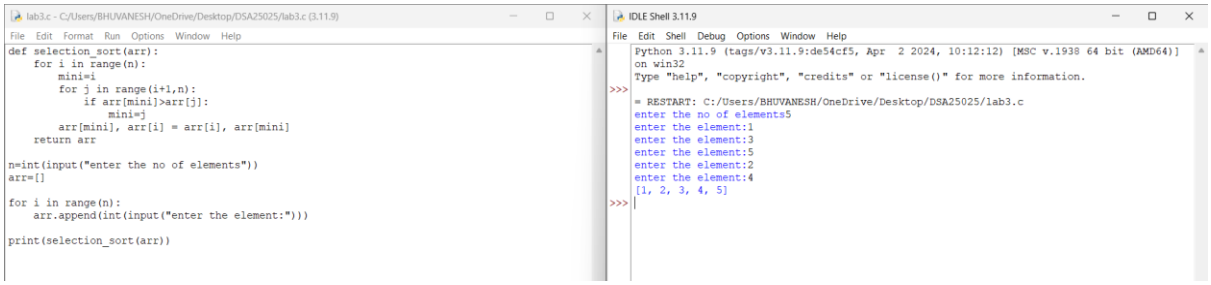
- After all elements are processed, the function returns the sorted array.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	Elements not sorted correctly	Shift elements before insertion

3c.write a python program to sort the elements using selection sort technique . take an input from the user.

Code and Output:



```
def selection_sort(arr):
    for i in range(n):
        mini=i
        for j in range(i+1,n):
            if arr[mini]>arr[j]:
                mini=j
        arr[mini], arr[i] = arr[i], arr[mini]
    return arr

n=int(input("enter the no of elements"))
arr=[]

for i in range(n):
    arr.append(int(input("enter the element:")))

print(selection_sort(arr))
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab3.c
enter the no of elements5
enter the element:1
enter the element:3
enter the element:5
enter the element:2
enter the element:4
[1, 2, 3, 4, 5]
>>>
```

Explanation:

Condition 1: for i in range(n) — Selecting Position

- The loop goes through each position in the array.
- i represents the current position where the smallest element will be placed.

Condition 2: mini = i — Assuming Minimum

- Initially, the element at index i is assumed to be the smallest.

Condition 3: for j in range(i+1,n) — Finding Minimum

- The loop checks all elements after position i.
- It searches for the smallest element.

Condition 4: if arr[mini] > arr[j] — Comparing

- Compares the current minimum with arr[j].
- If arr[j] is smaller, mini is updated.

Condition 5: mini = j — Updating Minimum

- Stores the index of the newly found smallest element.

Condition 6: arr[mini], arr[i] = arr[i], arr[mini] — Swapping

- The smallest element is swapped with the element at position i.
- This places the smallest element in its correct position.

Condition 7: return arr — Returning Result

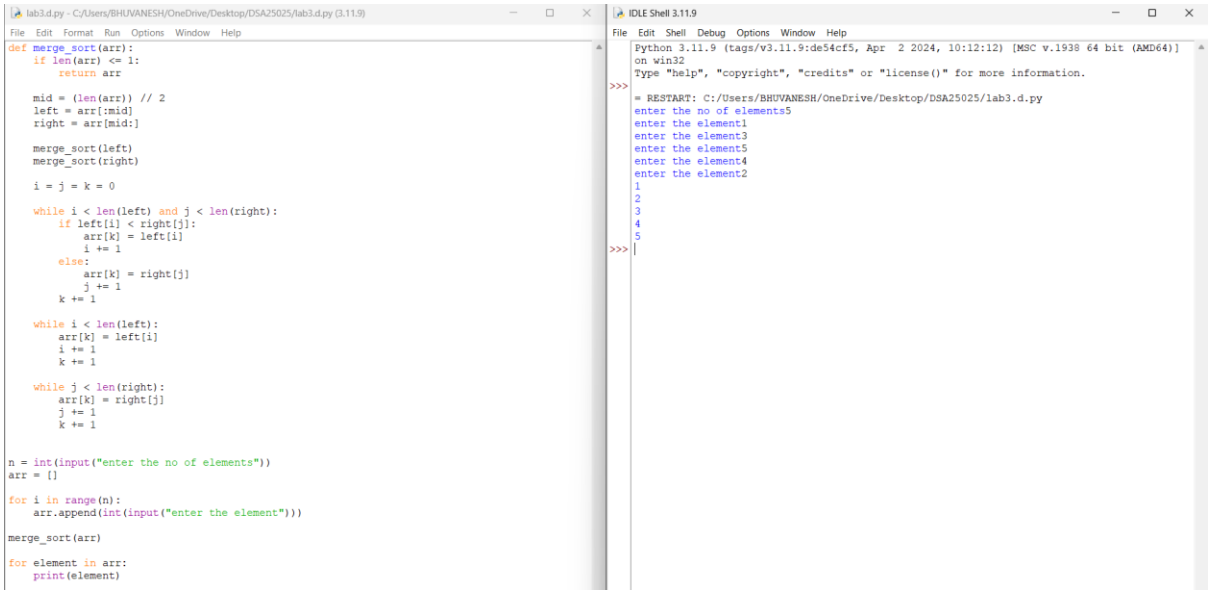
- Returns the completely sorted array.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	Wrong minimum value	Update min_index properly

3d.write a python program to sort the elements using the merge sort technique . take an input from the user.

Code and Output:



```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = (len(arr)) // 2
    left = arr[:mid]
    right = arr[mid:]

    merge_sort(left)
    merge_sort(right)

    i = j = k = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1


n = int(input("enter the no of elements"))
arr = []

for i in range(n):
    arr.append(int(input("enter the element")))

merge_sort(arr)

for element in arr:
    print(element)
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab3.d.py
enter the no of elements5
enter the element1
enter the element3
enter the element5
enter the element4
enter the element2
1
2
3
4
5
>>>
```

Explanation:

Condition 1: if len(arr) <= 1 — Checking the Base Condition

- The function first checks whether the array contains **one or zero elements**.

if len(arr) <= 1:

return arr

- If the array has one or zero elements, it is already sorted.
- No further division or comparison is required.
- The function returns the array as it is:

return arr

Condition 2: mid = len(arr) // 2 — Finding the Middle

- The array is divided into two parts.
- The middle position is calculated using:

mid = len(arr) // 2

- len(arr) gives the total number of elements.
- // 2 performs integer division and finds the middle index.

Condition 3: left = arr[:mid] — Creating the Left Part

- The elements before the middle position are stored in left.

left = arr[:mid]

Condition 4: right = arr[mid:] — Creating the Right Part

- The elements from the middle position to the end are stored in right.

right = arr[mid:]

Condition 5: merge_sort(left) and merge_sort(right) — Sorting Both Parts

- The left and right parts are recursively sorted.

merge_sort(left)

merge_sort(right)

- The array continues to be divided until each part contains only **one element**.
- A single element is already sorted.
- The smaller sorted parts are then merged together.

Condition 6: i = j = k = 0 — Initializing Indexes

- Three variables are initialized:

i = j = k = 0

- i keeps track of the current element in the **left array**.
- j keeps track of the current element in the **right array**.
- k keeps track of the position in the **original array** where the element will be placed.

Condition 7: while i < len(left) and j < len(right) — Comparing Both Parts

- The program compares elements from the left and right arrays.

while i < len(left) and j < len(right):

- The loop continues as long as both arrays still have elements to compare.
- This allows the program to select the smaller element and place it into the original array.

Condition 8: if left[i] < right[j] — Comparing Elements

- The current elements of the left and right arrays are compared.

if left[i] < right[j]:

- If the left element is smaller than the right element, it is placed into the original array.

arr[k] = left[i]

- Then i is increased:

i += 1

- This moves to the next element of the left array.

Condition 9: else — Taking the Right Element

- If the right element is smaller than or equal to the left element, the right element is placed into the original array.

else:

arr[k] = right[j]

- Then j is increased:

j += 1

- This moves to the next element of the right array.

Condition 10: k += 1 — Moving to the Next Position

- After placing an element into the original array, k is increased.

k += 1

Condition 11: while i < len(left) — Copying Remaining Left Elements

- Sometimes the right array becomes empty before the left array.

while i < len(left):

- Any remaining elements in the left array are copied into the original array.

arr[k] = left[i]

- Then both i and k are increased:

i += 1

k += 1

- The remaining elements are already sorted, so no comparison is needed.

Condition 12: while j < len(right) — Copying Remaining Right Elements

- Sometimes the left array becomes empty before the right array.

while j < len(right):

- Any remaining elements in the right array are copied into the original array.

arr[k] = right[j]

- Then j and k are increased:

j += 1

k += 1

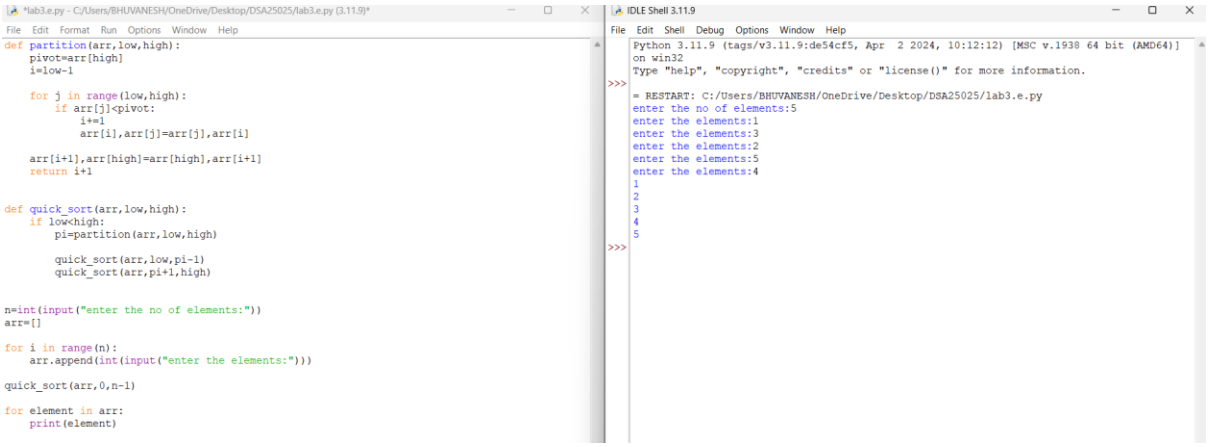
- This completes the merging process.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	Recursive error	Use if len(arr) <= 1

3e. Aim. Write a python program to sort the elements using a quick sort technique . take an input elements from the user

Code and output:



```
def partition(arr,low,high):
    pivot=arr[high]
    i=low-1

    for j in range(low,high):
        if arr[j]<pivot:
            i+=1
            arr[i],arr[j]=arr[j],arr[i]

    arr[i+1],arr[high]=arr[high],arr[i+1]
    return i+1


def quick_sort(arr,low,high):
    if low<high:
        pi=partition(arr,low,high)

        quick_sort(arr,low,pi-1)
        quick_sort(arr,pi+1,high)


n=int(input("enter the no of elements:"))
arr=[]

for i in range(n):
    arr.append(int(input("enter the elements:")))

quick_sort(arr,0,n-1)

for element in arr:
    print(element)
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab3.e.py
enter the no of elements:5
enter the elements:1
enter the elements:3
enter the elements:2
enter the elements:5
enter the elements:4
1
2
3
4
5
>>>
```

Explanation

Condition 1: if arr[j] < pivot — Element Smaller Than Pivot

- The function partition(arr, low, high) is used to divide the array into two parts.
- pivot = arr[high] selects the **last element** of the array as the pivot.
- i = low - 1 is initialized to keep track of the position where elements smaller than the pivot should be placed.
- The for loop checks each element from index low to high-1.
- The condition arr[j] < pivot checks whether the current element is smaller than the pivot.
- If the condition is **True**, i is increased by 1.
- The current element is then swapped with the element at index i:

arr[i], arr[j] = arr[j], arr[i]

- This places all elements smaller than the pivot toward the left side.

Condition 2: arr[j] >= pivot — No Swap Required

- If arr[j] < pivot is **False**, it means the current element is greater than or equal to the pivot.
- No swapping is performed.
- The loop continues to check the next element.
- These elements will remain on the right side of the pivot after partitioning.

Condition 3: arr[i+1], arr[high] = arr[high], arr[i+1] — Placing the Pivot

- After all elements have been checked, the pivot is placed in its correct position.
- The pivot is swapped with the element at index i+1:

arr[i+1], arr[high] = arr[high], arr[i+1]

- After this operation:
  - Elements smaller than the pivot are on the **left**.
  - The pivot is in its **correct position**.
  - Elements greater than or equal to the pivot are on the **right**.
- The function returns the pivot position:

return i+1

Condition 4: if low < high — Continue Quick Sort

- The function quick_sort(arr, low, high) is used to sort the array.
- The condition low < high checks whether the current part of the array contains more than one element.
- If the condition is **True**, the array needs to be sorted.
- The partition() function is called:

pi = partition(arr, low, high)

- pi stores the final position of the pivot.

Condition 5: quick_sort(arr, low, pi-1) — Sort Left Part

- After partitioning, the elements on the left side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, low, pi-1)

- The pivot itself is not included because it is already in its correct position.

Condition 6: quick_sort(arr, pi+1, high) — Sort Right Part

- The elements on the right side of the pivot are sorted separately.
- This is done using:

quick_sort(arr, pi+1, high)

- Again, the pivot is not included because it is already correctly positioned.

Input Section

- n = int(input("enter the no of elements:")) takes the number of elements from the user.
- arr = [] creates an empty list to store the elements.
- The for loop runs n times and takes each element from the user:
- for i in range(n):

arr.append(int(input("enter the elements:")))

- Each entered value is converted into an integer and added to the array using append().

Calling the Quick Sort Function

quick_sort(arr, 0, n-1)

- This calls the quick_sort() function.
- 0 represents the first index of the array.
- n-1 represents the last index of the array.
- The complete array is therefore passed for sorting.

Error table:

S.no	ERROR OBTAINED	RECTIFICTION
1	Wrong swapping conditions	By shifting to the correct spaces

WEEK-4

4a) AIM: write a python program to implement a single linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x. exit

code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        new_node.next = self.head
        self.head = new_node
        print(f"Inserted {data} at the beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return
```

Ln 232 Col 31

```
        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        print(f"Inserted {data} at the end.")

    # 4. Insert at Index
    def insert_at_index(self):
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
                print("Index out of range")
                return
            temp = temp.next

        if temp is None:
            print("Index out of range")
            return

        new_node = Node(data)

        new_node.next = temp.next
        temp.next = new_node
        print(f"Inserted {data} at index {index}.")

    # 5. Delete from Beginning
    def delete_beginning(self):

        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next
```

Ln: 2 Col: 29

```
        self.head = self.head.next
        print(f"Deleted beginning node.")

    # 6. Delete from End
    def delete_end(self):

        if self.head is None:
            print("List is empty")
            return

        if self.head.next is None:
            self.head = None
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None
        print(f"Deleted last node.")

    # 7. Delete from Index
    def delete_at_index(self):

        index = int(input("Enter index: "))

        if self.head is None:
            print("List is empty")
            return

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.head = self.head.next
            return

        temp = self.head

        for i in range(index - 1):
            if temp.next is None:
                print("Index out of range")
                return
            temp = temp.next
```

Ln: 122 Col: 0

```
            return
        temp = temp.next

    if temp.next is None:
        print("Index out of range")
        return

    temp.next = temp.next.next
    print(f"Deleted the node value.")

# 8. Count Nodes
def count_nodes(self):

    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)
# 9. Display
def display(self):

    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next

    print("None")



# Main Program

sll = SLL()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
```

Ln: 97 Col: 34

```
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()

    elif choice == 3:
        sll.insert_end()

    elif choice == 4:
        sll.insert_at_index()

    elif choice == 5:
        sll.delete_beginning()

    elif choice == 6:
        sll.delete_end()

    elif choice == 7:
        sll.delete_at_index()

    elif choice == 8:
        sll.count_nodes()

    elif choice == 9:
        sll.display()

    elif choice == 10:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

Ln: 142 Col: 28

Output:

Creating a linked list

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 10
Inserted 10 at the beginning.
```

Insertion at the end:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 22
Inserted 22 at the end.
```

Insertion at specific index:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter data: 15
Enter index: 3
Inserted 15 at index 3.
```

Delete at beginning:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5
Deleted beginning node.
```

Delete at the end:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6
Deleted last node.
```

Delete at the specific index:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 3
Deleted the node value.
```

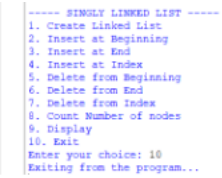
Number of nodes:

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 5
```

Display all the nodes:



Exiting the program:



Explanation:

1. A **singly linked list** is a collection of nodes where each node contains **two parts**: data and a link to the next node.
2. Each node contains:
   - **Data** – stores the value.
   - **Next (next)** – stores the reference/address of the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are created one by one and added to the list.
7. For the first node, both **head and tail** point to the same node, and the next of the tail is None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node's next points to the current head, and the new node becomes the new head.
9. During **insertion at the end**, the new node is added after the current tail. The current tail's next points to the new node, and the new node becomes the new tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the appropriate nodes by adjusting their next links.

11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by making the previous node point to the node after the deleted node.
12. If the node to be deleted is the **head**, the next node becomes the new head.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
16. During **deletion at the end**, the list is traversed to find the node before the tail. That node becomes the new tail and its next is set to None.
17. The **count operation** traverses the list from the head to the end and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves through the list using the next link until None is reached.
20. The program continues to execute until the user selects the **Exit** option.

Error table:

sno	Error obtained	rectification
1	Infinite loop or incomplete traversal due to not updating current	Move to the next node using temp = temp.next
2	Forgetting to update head while inserting/deleting at the beginning	Update head to point to the new first node using head = new_node or head = head.next

4b)AIM: write a python program to implement a double linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x. exit

code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoubleLinkedList:

    def __init__(self):
        self.head = None

    # a. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp

        print("Double Linked List created successfully.")

    # b. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        new_node.next = self.head

        if self.head is not None:
            self.head.prev = new_node

        self.head = new_node

        print(f"Inserted {data} at the beginning.")
```

```
        print(f"Inserted {data} at the beginning.")

    # c. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print(f"Inserted {data} at the end.")

    # d. Insert at Specific Index
    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))

        if index < 0:
            print("Invalid index.")
            return

        new_node = Node(data)

        if index == 0:
            new_node.next = self.head

            if self.head is not None:
                self.head.prev = new_node

            self.head = new_node

            print(f"Inserted {data} at index {index}.")
            return

        temp = self.head

        for i in range(index - 1):
```

```
        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return

            temp = temp.next

        if temp is None:
            print("Index out of range.")
            return

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print(f"Inserted {data} at index {index}.")

    # e. Delete by Value
    def delete_by_value(self):
        value = int(input("Enter value to delete: "))

        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        temp = self.head

        while temp is not None:
            if temp.data == value:
                break
            temp = temp.next

        if temp is None:
            print(f"{value} not found in the Linked List.")
            return

        if temp.prev is None:
            self.head = temp.next

            if self.head is not None:
                self.head.prev = None
```

```
                self.head.prev = None

        else:
            temp.prev.next = temp.next

            if temp.next is not None:
                temp.next.prev = temp.prev

        print(f"Deleted {value}.")

    # f. Delete First Node
    def delete_first(self):
        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        deleted = self.head.data

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print(f"Deleted first node: {deleted}")

    # g. Delete Last Node
    def delete_last(self):
        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        temp = self.head

        if temp.next is None:
            deleted = temp.data
            self.head = None
            print(f"Deleted last node: {deleted}")
            return

        while temp.next is not None:
            temp = temp.next

        deleted = temp.data

        temp.prev.next = None

        print(f"Deleted last node: {deleted}")
```

```
        print(f"Deleted last node: {deleted}")

    # h. Count Number of Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # i. Display
    def display(self):
        if self.head is None:
            print("Doubly Linked List is empty.")
            return

        temp = self.head

        print("Doubly Linked List:", end=" ")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")


# Main Program
dll = DoubleLinkedList()

while True:

    print("\n----- DOUBLE LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display")
    print("10. Exit")
```

```
print("\n----- DOUBLE LINKED LIST -----")
print("1. Create Linked List")
print("2. Insert at Beginning")
print("3. Insert at End")
print("4. Insert at Specific Index")
print("5. Delete by Value")
print("6. Delete First Node")
print("7. Delete Last Node")
print("8. Count Number of Nodes")
print("9. Display")
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    dll.create()

elif choice == 2:
    dll.insert_beginning()

elif choice == 3:
    dll.insert_end()

elif choice == 4:
    dll.insert_at_index()

elif choice == 5:
    dll.delete_by_value()

elif choice == 6:
    dll.delete_first()

elif choice == 7:
    dll.delete_last()

elif choice == 8:
    dll.count_nodes()

elif choice == 9:
    dll.display()

elif choice == 10:
    print("Exiting Program...")
    break

else:
    print("Invalid choice. Please try again.")
```

Ln: 183 Col: 0

Output:

Inserting at the beginning:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 2
Inserted 2 at the beginning.
```

Insert at the end:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 11
Inserted 11 at the end.
```

Insert at the specific index:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 6
Enter data: 6
Inserted 6 at index 6.
```

Delete By value

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 5
Enter value to delete: 3
Deleted 3.
```

Delete at the beginning:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 6
Deleted first node: 2
```

Delete at the end:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 7
Deleted last node: 11
```

Number of nodes:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 5
```

Display the nodes:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 9
Doubly Linked List: 1 <-> 5 <-> 7 <-> 9 <-> 6 <-> None
```

Exiting the program:

```
----- DOUBLE LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 10
Exiting Program...
>>>
```

Explanation:

1. A **doubly linked list** is a collection of nodes where each node contains **three parts**: data, a link to the previous node, and a link to the next node.
2. Each node has:
   - **Data** – stores the value.
   - **Previous (prev)** – points to the previous node.
   - **Next (next)** – points to the next node.
3. The program maintains two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and maintain the data, prev, and next references.
6. During **creation of the linked list**, the user enters the number of nodes and their data. The nodes are added one by one at the end of the list.
7. For the first node, both **head and tail** point to the same node. The prev of the head and the next of the tail are None.
8. During **insertion at the beginning**, a new node is created and placed before the current head. The new node becomes the head, its next points to the previous head, and the previous head's prev points back to the new node.
9. During **insertion at the end**, the new node is placed after the current tail. The new node's prev points to the old tail, the old tail's next points to the new node, and the new node becomes the tail.
10. During **insertion at a specific index**, the list is traversed until the required position is reached. The new node is inserted between the previous and next nodes, and both prev and next links are updated.
11. During **deletion by value**, the list is searched for the node containing the given value. Once found, its previous and next nodes are connected to each other, removing the selected node from the list.
12. If the node to be deleted is the **head**, the next node becomes the new head and its prev is set to None.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is set to None.
14. If the list contains only one node and that node is deleted, both **head and tail are set to None**.
15. During **deletion at the beginning**, the head node is removed and the next node becomes the new head. The new head's prev is set to None.

16. During **deletion at the end**, the tail node is removed and the previous node becomes the new tail. The new tail's next is set to None.
17. The **count operation** traverses the list from the head to the tail and counts the total number of nodes.
18. The **display/traverse operation** starts from the head and moves forward using the next link to display all the nodes.
19. Since it is a doubly linked list, the program can also traverse the list **backwards**, starting from the tail and using the prev links.
20. The program displays the **head and tail** values to show the starting and ending nodes of the list.
21. The important relationships maintained in a doubly linked list are:

head.prev = None

tail.next = None

22. The overall structure of a doubly linked list is:

None ← [10] ⇄ [20] ⇄ [30] ⇄ [40] → None

↑ ↑

HEAD TAIL

23. The arrows in both directions allow the list to be traversed **forward as well as backward**.
24. The program is **menu-driven**, allowing the user to perform creation, insertion, deletion, counting, displaying, and other operations repeatedly until the **Exit** option is selected.

Error table:

Sno	Error obtained	Rectification
1	Forgetting to update the prev pointer	Update both prev and next pointers whenever a node is inserted
2	Incorrect head.prev	Maintain head.prev = None

4c)AIM: write a python program to implement a circular linked list which could perform the below operations by taking an input from the user

I .create a linked list

ii. insert at beginning

iii. insert at end

iv. insert at specific index

v. delete by value

vi. delete at beginning

vii. delete at the end

viii. count no of nodes

ix. display or travsere

x: display the head and tail

xi. exit

code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # i. Create a linked list
    def create(self):
        n = int(input("Enter the number of nodes: "))

        if n <= 0:
            print("List cannot be created.")
            return

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            self.insert_end(data)

        print("Circular linked list created successfully.")

    # ii. Insert at beginning
    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head
        else:
            new_node.next = self.head
            self.head = new_node
            self.tail.next = self.head

        print("Node inserted at beginning.")

    # iii. Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            self.tail = new_node
```

Ln: 270 Col: 50

```
            self.tail.next = new_node
            self.tail = new_node

        print("Node inserted at end.")

    # iv. Insert at specific index
    def insert_at_index(self, data, index):
        if index < 0:
            print("Invalid index.")
            return

        if index == 0:
            self.insert_beginning(data)
            return

        if self.head is None:
            print("List is empty.")
            return

        new_node = Node(data)
        current = self.head
        count = 0

        while count < index - 1 and current != self.tail:
            current = current.next
            count += 1

        if count != index - 1:
            print("Index out of range.")
            return

        new_node.next = current.next
        current.next = new_node

        if current == self.tail:
            self.tail = new_node

        print("Node inserted at index", index)

    # v. Delete by value
    def delete_by_value(self, value):
        if self.head is None:
            print("List is empty.")
            return

        # If head contains the value
        if self.head.data == value:
```

```
        # If head contains the value
        if self.head.data == value:

            # Only one node
            if self.head == self.tail:
                self.head = None
                self.tail = None
            else:
                self.head = self.head.next
                self.tail.next = self.head

            print("Node deleted.")
            return

        current = self.head

        while current.next != self.head:
            if current.next.data == value:

                # If deleting tail
                if current.next == self.tail:
                    self.tail = current

                current.next = current.next.next
                self.tail.next = self.head

                print("Node deleted.")
                return

            current = current.next

        print("Value not found.")

    # vi. Delete at beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty.")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
        else:
            self.head = self.head.next
            self.tail.next = self.head

        print("First node deleted.")
```

```
        print("First node deleted.")

    # vii. Delete at end
    def delete_end(self):
        if self.head is None:
            print("List is empty.")
            return

        # Only one node
        if self.head == self.tail:
            self.head = None
            self.tail = None
            print("Last node deleted.")
            return

        current = self.head

        while current.next != self.tail:
            current = current.next

        current.next = self.head
        self.tail = current

        print("Last node deleted.")

    # viii. Count number of nodes
    def count_nodes(self):
        if self.head is None:
            return 0

        count = 1
        current = self.head.next

        while current != self.head:
            count += 1
            current = current.next

        return count

    # ix. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        current = self.head
```

```
            return

        current = self.head

        print("Circular Linked List:")

        while True:
            print(current.data, end=" -> ")
            current = current.next

            if current == self.head:
                break

        print("(HEAD)")

        print("Head:", self.head.data)
        print("Tail:", self.tail.data)
        print("Tail.next:", self.tail.next.data, "(Head)")


# Main program
cll = CircularLinkedList()

while True:

    print("\n========== CIRCULAR LINKED LIST ==========")
    print("1. Create a linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete at beginning")
    print("7. Delete at the end")
    print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Display Head and Tail")
    print("11. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        cll.create()
```

```
if choice == 1:
    cll.create()

elif choice == 2:
    data = int(input("Enter data: "))
    cll.insert_beginning(data)

elif choice == 3:
    data = int(input("Enter data: "))
    cll.insert_end(data)

elif choice == 4:
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))
    cll.insert_at_index(data, index)

elif choice == 5:
    value = int(input("Enter value to delete: "))
    cll.delete_by_value(value)

elif choice == 6:
    cll.delete_beginning()

elif choice == 7:
    cll.delete_end()

elif choice == 8:
    print("Number of nodes:", cll.count_nodes())

elif choice == 9:
    cll.display()

elif choice == 10:
    if cll.head is None:
        print("List is empty.")
    else:
        print("Head:", cll.head.data)
        print("Tail:", cll.tail.data)
        print("Tail.next:", cll.tail.next.data, "(Head)")

elif choice == 11:
    print("Program exited.")
    break

else:
    print("Invalid choice. Please try again.")
```

Output:

Creating a linked list:

```
= RESTART: C:/Users/shari/AppData/Local/Programs/Python/Python313/circularlinkedlist.py

========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 1
Enter the number of nodes: 5
Enter data for node 1: 2
Node inserted at end.
Enter data for node 2: 4
Node inserted at end.
Enter data for node 3: 6
Node inserted at end.
Enter data for node 4: 8
Node inserted at end.
Enter data for node 5: 10
Node inserted at end.
Circular linked list created successfully.
```

Inserting at the beginning:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 2
Enter data: 1
Node inserted at beginning.
```

Insert at the end:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 3
Enter data: 5
Node inserted at end.
```

Insert at the specific index:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 4
Enter data: 7
Enter index: 4
Node inserted at index 4
```

Delete by value:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 5
Enter value to delete: 6
Node deleted.
```

Delete at the beginning:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 6
First node deleted.
```

Delete at the end:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 7
Last node deleted.
```

Number of nodes:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 8
Number of nodes: 5
```

Display all the nodes:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 9
Circular Linked List:
2 -> 4 -> 7 -> 8 -> 10 -> (HEAD)
Head: 2
Tail: 10
Tail.next: 2 (Head)
```

Display the head and the tail

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 10
Head: 2
Tail: 10
Tail.next: 2 (Head)
```

Exit: the program:

```
========== CIRCULAR LINKED LIST ==========
1. Create a linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete at beginning
7. Delete at the end
8. Count number of nodes
9. Display / Traverse
10. Display Head and Tail
11. Exit
Enter your choice: 11
Program exited.
```

Explanation :

1. A **circular singly linked list** is a collection of nodes where each node contains **data** and a **link to the next node**.
2. In a circular linked list, the **last node points back to the first node (head)** instead of pointing to None.
3. The program uses two references:
   - **Head** – points to the first node.
   - **Tail** – points to the last node.
4. Initially, when the list is empty, both head and tail are set to None.
5. A **Node class** is used to create individual nodes and store the data and reference to the next node.
6. During **creation of the linked list**, the user enters the number of nodes and their data. Each new node is inserted at the end of the list.
7. If the list is empty, the first node becomes both the **head and tail**, and its next points back to itself.
8. During **insertion at the beginning**, the new node is placed before the current head. The new node becomes the head, and the tail is connected to the new head.
9. During **insertion at the end**, the new node is added after the current tail. The new node becomes the tail, and its next points to the head.

10. During **insertion at a specific index**, the list is traversed until the required position is reached, and the new node is inserted between two existing nodes.
11. During **deletion by value**, the list is searched for the node containing the given value. The node is removed by connecting its previous node directly to its next node.
12. If the node to be deleted is the **head**, the next node becomes the new head and the tail is connected to it.
13. If the node to be deleted is the **tail**, the previous node becomes the new tail and its next is connected to the head.
14. During **deletion at the beginning**, the head node is removed and the next node becomes the new head.
15. During **deletion at the end**, the node before the tail is found and made the new tail.
16. The **count operation** traverses the list and counts all the nodes. Traversal stops when the head is reached again.
17. The **display/traverse operation** starts from the head and visits every node until it reaches the head again.
18. Since the list is circular, traversal does not use current != None. Instead, it stops when current becomes equal to head.
19. The program also displays the **head, tail, and tail.next** to show the circular connection.
20. The program is **menu-driven**, so the user can repeatedly perform different operations until the **Exit** option is selected.
21. The most important property maintained throughout the program is:

tail.next = head

22. Thus, the structure of the circular linked list is:

HEAD → Node → Node → Node → TAIL

↑ |

|_____|

This ensures that the linked list remains **circular after every insertion and deletion operation**.

Error table:

sno	Error obtained	Rectification
1	Using temp != None as the traversal condition	Stop traversal when temp == head

2	Forgetting to update tail.next when the head changes	After changing the head, update tail.next = head

WEEK-5:

5A.

Aim: write a python program to implement the stack using the arrays. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:

```
class Stack:
    def __init__(self):
        self.stack = []

    # Push operation
    def push(self, data):
        self.stack.append(data)
        print(data, "pushed into the stack.")

    # Pop operation
    def pop(self):
        if len(self.stack) == 0:
            print("Stack Underflow. Stack is empty.")
        else:
            data = self.stack.pop()
            print(data, "popped from the stack.")

    # Peek operation
    def peek(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element is:", self.stack[-1])

    # Display all elements
    def display(self):
        if len(self.stack) == 0:
            print("Stack is empty.")
        else:
            print("Stack elements are:")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i])

    # Count number of elements
    def count(self):
        print("Number of elements:", len(self.stack))

# Main program
s = Stack()

while True:
    print("\n--- STACK MENU ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
```

```
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        s.count()

    elif choice == 6:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

Pushing an element into the stack

```
======== RESTART: C:\Users\BHUVANESH\OneDrive\Desktop\DSA25025\lab5a.py ========

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 5
5 pushed into the stack.
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 20
20 pushed into the stack.
```

Popping an element from the stack

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 2
20 popped from the stack.
```

Peek element

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 3
Top element is: 10
```

Display all the elements in the stack

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Stack elements are:
10
```

Count

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 5
Number of elements: 1
```

Exiting the program

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Program ended.
```

Explanation:

1. Class and Initialization

- A class named Stack is created.
- The __init__() method acts as the constructor.
- An empty list self.stack is created to store stack elements.
- The stack follows the **LIFO (Last In, First Out)** principle.

2. Push Operation

- The push() method is used to insert an element into the stack.
- The append() function adds the element to the end of the list.
- The newly inserted element becomes the **top element**.
- A message is displayed after the element is inserted.

3. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether the stack is empty.
- If the stack is empty, **Stack Underflow** is displayed.
- Otherwise, the pop() function removes the last element.
- The removed element is displayed.

4. Peek Operation

- The peek() method is used to view the top element.

- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.stack[-1] is used to access the top element.
- The top element is displayed without removing it.

5. Display Operation

- The display() method is used to display all stack elements.
- The program first checks whether the stack is empty.
- If empty, **Stack is empty** is displayed.
- Otherwise, a for loop is used to traverse the stack.
- The elements are displayed from **top to bottom**.

6. Count Operation

- The count() method is used to find the number of elements in the stack.
- The len() function returns the number of elements.
- The count is displayed to the user.

Error table:

sno	Error obtained	Error rectification
1	Stack underflow	Check whether the stack is empty before poping
2	Name error	Define the class before creating the object

5b.

Aim: write a python program to implement the stack using the linked list. And also implement stack operations such as push, pop, peek and also display the stack elements.

Code:

```python
# Node class
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Stack class
class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new_node = Node(data)

        new_node.next = self.top
        self.top = new_node

        print(data, "pushed into the stack.")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow. Stack is empty.")
        else:
            data = self.top.data
            self.top = self.top.next

            print(data, "popped from the stack.")

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            print("Top element is:", self.top.data)

    # Display all elements
    def display(self):
        if self.top is None:
            print("Stack is empty.")
        else:
            print("Stack elements are:")

            current = self.top
```

```python
    # Count number of elements
    def count(self):
        count = 0
        current = self.top

        while current is not None:
            count += 1
            current = current.next

        print("Number of elements:", count)
# Main program
s = Stack()
while True:
    print("\n--- STACK MENU ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Count")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.peek()

    elif choice == 4:
        s.display()

    elif choice == 5:
        s.count()

    elif choice == 6:
        print("Program ended.")
        break

    else:
        print("Invalid choice.")
```

Output:

Pushing an element into the stack

```
========== RESTART: C:\Users\BHUVANESH\OneDrive\Desktop\DSA25025\lab5b.py ==========

----- LINKED LIST MENU -----
1. Insert at Beginning
2. Insert at End
3. Delete from Beginning
4. Delete from End
5. Display
6. Count
7. Exit
Enter your choice: 1
Enter the element: 10
10 inserted at beginning.
```

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 1
Enter the element: 20
20 pushed into the stack.
```

Poping an element from the stack

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 2
20 popped from the stack.
```

Peek element

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 3
Top element is: 10
```

Display all the elements in the stack

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 4
Stack elements are:
10
```

count

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 5
Number of elements: 1
```

Exiting the program

```
--- STACK MENU ---
1. Push
2. Pop
3. Peek
4. Display
5. Count
6. Exit
Enter your choice: 6
Program ended.
```

Explanation:

1. Node Class

- A class named Node is created to represent each element of the linked list.
- The __init__() method initializes a node.
- self.data stores the value of the node.
- self.next stores the reference to the next node.
- Initially, self.next is set to None.

2. Stack Class

- A class named Stack is created to implement the stack operations.
- The __init__() method initializes the stack.
- self.top = None indicates that the stack is initially empty.
- The top variable stores the reference to the top node of the stack.

3. Push Operation

- The push() method is used to insert an element into the stack.
- A new node is created using Node(data).
- The new node is connected to the current top node using:

new_node.next = self.top

- The new node is then made the top node using:

self.top = new_node

- The inserted element is displayed.
- The operation follows the **LIFO (Last In, First Out)** principle.

4. Pop Operation

- The pop() method is used to remove the top element.
- The program first checks whether self.top is None.
- If self.top is None, the stack is empty and **Stack Underflow** is displayed.
- Otherwise, the data of the top node is stored in data.
- The top is moved to the next node using:

self.top = self.top.next

- The removed element is displayed.

5. Peek Operation

- The peek() method is used to view the top element without removing it.
- The program checks whether the stack is empty.
- If the stack is empty, **Stack is empty** is displayed.
- Otherwise, self.top.data is used to display the top element.
- The stack remains unchanged after the peek operation.

6. Display Operation

- The display() method is used to display all the elements of the stack.
- The program first checks whether the stack is empty.
- A temporary variable current is initialized with self.top.
- A while loop is used to traverse the linked list.
- current.data displays the value of each node.
- current = current.next moves to the next node.
- The elements are displayed from **top to bottom**.

7. Count Operation

- The count() method is used to find the number of elements in the stack.
- A variable count is initialized to 0.
- A temporary variable current starts from self.top.
- The linked list is traversed using a while loop.
- The count value is increased for every node.

- The total number of elements is displayed.

Error table:

sno	Error obtained	Error rectification
1	Incorrect push	Set new_node.next = self.top before updating self.top
2	Infinte loop	Use break when choice == 6

WEEK – 6

6. AIM: To implement Queue and its operations such as Enqueue, Dequeue, Peek, Display with a menu driven approach.

6a. Using Array, Linked list, Data structures.

6b. Circular Queue.

PROGRAM:

```
File  Edit  Format  Run  Options  Window  Help
MAX = 5
queue = []
while True:
    print("\n--- QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(queue) == MAX:
            print("Queue Overflow")
        else:
            value = int(input("Enter value: "))
            queue.append(value)
            print(value, "inserted")

    elif choice == 2:
        if len(queue) == 0:
            print("Queue Underflow")
        else:
            value = queue.pop(0)
            print(value, "deleted")

    elif choice == 3:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Front element =", queue[0])

    elif choice == 4:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Queue elements:", queue)

    elif choice == 5:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

```
>>
======= RESTART: C:/Users/OMEN/OneDrive/Desktop/dsa git/week 6/week6.1.py ==

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Queue is empty

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 4
4 inserted

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: [4]

--- QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

- What is a Queue?
  - A Queue is a linear data structure that follows the FIFO (First In First Out) principle.
  - Elements are inserted at the rear and removed from the front.

Arrays:

- **Components in the Code**
  1. **Initialization (__init__)**
     - Creates a queue of fixed size.
     - Uses an array (self.queue) to store elements.

- front and rear pointers track the first and last elements.
- Both start at -1 (empty queue).

2. **Enqueue (Insertion)**
   - Adds an element at the rear.
   - If rear == size - 1, → **Overflow** (queue full).
   - If queue is empty (front == -1), set front = 0.
   - Increment rear and insert the element.
3. **Dequeue (Deletion)**

- Removes an element from the front.
- If queue is empty (front == -1 or front > rear), → **Underflow**.
- Deletes the element at front, increments front.
- If all elements are deleted (front > rear), reset both pointers to -1.

4. **Peek**

- Shows the element at the front without removing it.
- If empty, prints "Queue is empty".

5. **Display**

- Prints all elements between front and rear.
- If empty, prints "Queue is empty".

◇ **Menu-Driven Program**

- User chooses operations from a menu:
  1. **Enqueue** → Insert element.
  2. **Dequeue** → Remove element.
  3. **Peek** → Show front element.
  4. **Display** → Show all elements.
  5. **Exit** → End program.
- The loop continues until the user selects **Exit**.

◇ **Characteristics of This Queue**

- It is a **Linear Queue** (not circular).
- **Overflow** occurs when rear == size - 1.
- **Underflow** occurs when queue is empty.
- Once rear reaches the end, no more insertions are possible even if earlier slots are freed (limitation of linear queue).

Linked List:

◇ **Structure in the Code**

1. **Initialization (__init__)**
   - Creates a queue of fixed size (default = 5).

- Uses an array (self.queue) to store elements.
- front and rear pointers track the first and last elements.
- Both start at -1 (empty queue).

2. **Enqueue (Insertion)**
   - Adds an element at the rear.
   - If rear == size - 1, → **Overflow** (queue full).
   - If queue is empty (front == -1), set front = 0.
   - Increment rear and insert the element.
3. **Dequeue (Deletion)**
   - Removes an element from the front.
   - If queue is empty (front == -1), → **Underflow**.
   - Deletes the element at front, increments front.
   - If all elements are deleted (front > rear), reset both pointers to -1.
4. **Peek**
   - Shows the element at the front without removing it.
   - If empty, prints "Queue is empty".
5. **Display**
   - Prints all elements between front and rear.
   - If empty, prints "Queue is empty".

◇ **Characteristics of This Queue**

- It is a **Linear Queue** (not circular).
- **Overflow** occurs when rear == size - 1.
- **Underflow** occurs when queue is empty (front == -1).
- Once rear reaches the end, no more insertions are possible even if earlier slots are freed (limitation of linear queue).

6b. To implement Circular Queue and its operation**.**

PROGRAM:

week 6.2.py - C:/Users/OMEN/OneDrive/Desktop/dsa git/week 6/week 6.2.py (3.14.0)

File Edit Format Run Options Window Help

```
MAX = 5
queue = [None] * MAX
front = -1
rear = -1
while True:
    print("\n--- CIRCULAR QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:

        if (rear + 1) % MAX == front:
            print("Circular Queue Overflow")
        else:
            value = int(input("Enter value: "))

            if front == -1:
                front = 0
                rear = 0
            else:
                rear = (rear + 1) % MAX

            queue[rear] = value

            print(value, "inserted")

    elif choice == 2:

        if front == -1:
            print("Circular Queue Underflow")

        else:
            print(queue[front], "deleted")
            queue[front] = None
            if front == rear:
                front = -1
                rear = -1

            else:
                front = (front + 1) % MAX

    elif choice == 3:

        if front == -1:
            print("Circular Queue is empty")
        else:
            print("Front element =", queue[front])
```

```
elif choice == 4:

    if front == -1:
        print("Circular Queue is empty")

    else:
        print("Queue elements:", end=" ")

        i = front

        while True:
            print(queue[i], end=" ")

            if i == rear:
                break

            i = (i + 1) % MAX

        print()
elif choice == 5:
    print("Program ended")
    break
else:
    print("Invalid choice")
```

OUTPUT:

```
====== RESTART: C:/Users/OMEN/OneDrive/Desktop/dsa git/week 6/week 6.2.py ======

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Circular Queue Underflow

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 4
4 inserted

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value: 5
5 inserted

--- CIRCULAR QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
4 deleted

--- CIRCULAR QUEUE MENU ---
```

EXPLANATION:

- **Concept of Circular Queue**
  - A **Circular Queue** is a linear data structure that follows the **FIFO (First In First Out)** principle.
  - Unlike a simple linear queue, the last position connects back to the first position, forming a circle.
  - This design solves the problem of wasted space in a linear queue when elements are deleted from the front.
- **Structure in the Code**

1. **Initialization (__init__)**
    - Creates a queue of fixed size.
    - Uses an array (self.queue) to store elements.
    - front and rear pointers track the first and last elements.
    - Both start at -1 (empty queue).
2. **Enqueue (Insertion)**
    - Adds an element at the rear.
    - Overflow occurs if (rear + 1) % size == front (queue full).
    - If queue is empty (front == -1), both front and rear are set to 0.
    - Otherwise, rear moves forward using modulo arithmetic: (rear + 1) % size.

3. **Dequeue (Deletion)**

- Removes an element from the front.
- Underflow occurs if front == -1 (queue empty).
- If only one element is left (front == rear), both pointers reset to -1.
- Otherwise, front moves forward using (front + 1) % size.

4.**Display**

- Prints all elements from front to rear using modulo arithmetic.
- Traverses circularly until it reaches the rear.

◇ Menu-Driven Program

- User chooses operations from a menu:
    1. **Enqueue** → Insert element.
    2. **Dequeue** → Remove element.
    3. **Display** → Show all elements.
    4. **Exit** → End program.
- The loop continues until the user selects **Exit**.

◇ Characteristics of Circular Queue

- **Efficient use of memory**: Freed spaces are reused.
- **FIFO principle** maintained.
- **Overflow condition:** (rear + 1) % size == front.
- **Underflow condition:** front == -1.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Redundant underflow check → Condition if self.front == -1 or self.front > self.rear: is slightly redundant because resetting pointers already handles empty state.	Simplify to if self.front == -1.
2.	Missing underflow condition → In dequeue, only checks if self.front == -1. If multiple deletions occur, front > rear is not handled, leading to incorrect behavior.	Add condition: if self.front == -1 or self.front > self.rear.
3.	Circular Queue: Display traversal limitation → Display prints values but not indices, making debugging harder.	Optional improvement: print both index and value.

Week-7

AIM:write a python program to implement binary tree with all its operations by using arrays :

Code:

```
class BinaryTree:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

    def insert(self, value, parent=None, side=None):
        if parent is None:
            if self.tree[0] is None:
                self.tree[0] = value
                print("Root inserted successfully")
            else:
                print("Root already exists")
            return

        if parent not in self.tree:
            print("Parent not found")
            return

        i = self.tree.index(parent)

        if side == "L":
            child = 2 * i + 1
        elif side == "R":
            child = 2 * i + 2
        else:
            print("Invalid side")
            return

        if child >= self.size:
            print("Array size exceeded")
        elif self.tree[child] is None:
            self.tree[child] = value
            print("Node inserted successfully")
        else:
            print("Position already occupied")

    def search(self, value):
        if value in self.tree:
            print("Element found at index", self.tree.index(value))
        else:
            print("Element not found")

    def display(self):
        print("Index\tNode")
        for i in range(self.size):
            if self.tree[i] is not None:
                print(i, "\t", self.tree[i])

    def delete(self, value):
        if value not in self.tree:
```

```
            if self.tree[i] is not None:
                print(i, "\t", self.tree[i])

    def delete(self, value):
        if value not in self.tree:
            print("Element not found")
            return

        i = self.tree.index(value)
        left = 2 * i + 1
        right = 2 * i + 2

        if ((left < self.size and self.tree[left] is not None) or
                (right < self.size and self.tree[right] is not None)):
            print("Cannot delete: Node is not a leaf")
        else:
            self.tree[i] = None
            print("Leaf node deleted successfully")

    def preorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def inorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.inorder(2 * i + 1)
            print(self.tree[i], end=" ")
            self.inorder(2 * i + 2)

    def postorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")


n = int(input("Enter array size: "))
bt = BinaryTree(n)

while True:
    print("\n1.Insert Root")
    print("2.Insert Child")
    print("3.Search")
    print("4.Display")
    print("5.Delete Leaf Node")
    print("6.Preorder")
    print("7.Inorder")
    print("8.Postorder")
```

```
    print("5.Delete Leaf Node")
    print("6.Preorder")
    print("7.Inorder")
    print("8.Postorder")
    print("9.Exit")

    ch = int(input("Enter choice: "))

    if ch == 1:
        value = int(input("Enter root value: "))
        bt.insert(value)

    elif ch == 2:
        parent = int(input("Enter parent value: "))
        side = input("Enter side (L/R): ").upper()
        value = int(input("Enter child value: "))
        bt.insert(value, parent, side)

    elif ch == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)

    elif ch == 4:
        bt.display()

    elif ch == 5:
        value = int(input("Enter leaf value to delete: "))
        bt.delete(value)

    elif ch == 6:
        print("Preorder:", end=" ")
        bt.preorder()
        print()

    elif ch == 7:
        print("Inorder:", end=" ")
        bt.inorder()
        print()

    elif ch == 8:
        print("Postorder:", end=" ")
        bt.postorder()
        print()

    elif ch == 9:
        break

    else:
        print("Invalid choice")
```

Ln: 1 Col: 0

Output:

```
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
======== RESTART: C:\Users\BHUVANESH\OneDrive\Desktop\DSA25025\lab7a.py ========
Enter array size: 5

1.Insert Root
2.Insert Child
3.Search
4.Display
5.Delete Leaf Node
6.Preorder
7.Inorder
8.Postorder
9.Exit
Enter choice: 1
Enter root value: 5
Root inserted successfully
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 20
Enter left child value: 15
Parent not found!
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 10
Enter right child value: 15
Right child inserted successfully.
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 4
Enter element to search: 13
Element not found.
```

```
--- BINARY TREE MENU ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 7
Inorder: 10 15
```

Explanation:

1. A binary tree is a non-linear data structure in which each node can have at most two children: a left child and a right child.
2. In array representation, all nodes are stored in an array using index positions.
3. The root node is stored at index 0.
4. For a node at index i:
   - Left child: 2*i + 1
   - Right child: 2*i + 2
   - Parent: (i-1)//2, for i > 0.
5. Insertion: Inserts the root or a left/right child into the appropriate array position if it is empty.
6. Searching: Checks whether the given element exists in the array.
7. Display: Displays the tree elements along with their array indices.
8. Deletion: Deletes a leaf node by replacing its value with None.
9. Preorder traversal: Visits the root, left subtree, and right subtree.
10. Inorder traversal: Visits the left subtree, root, and right subtree.
11. Postorder traversal: Visits the left subtree, right subtree, and root.
12. Array representation provides direct index access but may waste memory for sparse trees.

2. Binary Tree Using a Linked List — Concept

1. A binary tree is a non-linear data structure in which each node has at most two children.
2. In linked-list representation, each node is created dynamically using a class.
3. Each node contains three components: data, left, and right.
4. The data field stores the value, while left and right store references to the child nodes.
5. The root variable points to the first node of the tree. If the tree is empty, the root is None.
6. Insertion: Creates a new node and connects it to the left or right of the specified parent node.
7. Searching: Uses a queue to search for the required element level by level.
8. Display: Displays all nodes in level-order traversal using a queue.
9. Deletion: Removes a leaf node by setting its parent's corresponding child reference to None.

10. Preorder traversal: Visits the root, left subtree, and right subtree.
11. Inorder traversal: Visits the left subtree, root, and right subtree.
12. Postorder traversal: Visits the left subtree, right subtree, and root.
13. Linked-list representation uses dynamic memory allocation and does not require a fixed array size.
14. It is suitable for irregularly shaped trees, but each node requires extra memory for references.

7b.

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert_root(self, value):
        if self.root is None:
            self.root = Node(value)
            print("Root inserted successfully.")
        else:
            print("Root already exists!")

    def find(self, value):
        if self.root is None:
            return None

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.data == value:
                return current

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        return None

    def insert_left(self, parent, value):
        node = self.find(parent)

        if node is None:
            print("Parent not found!")
        elif node.left is not None:
            print("Left child already exists!")
        else:
```

```
            print("Left child already exists!")
        else:
            node.left = Node(value)
            print("Left child inserted successfully.")

    def insert_right(self, parent, value):
        node = self.find(parent)

        if node is None:
            print("Parent not found!")
        elif node.right is not None:
            print("Right child already exists!")
        else:
            node.right = Node(value)
            print("Right child inserted successfully.")

    def search(self, value):
        if self.find(value):
            print("Element found.")
        else:
            print("Element not found.")

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    def display(self):
        if self.root is None:
            print("Tree is empty.")
            return

        print("Level-order display:")
        queue = [self.root]
```

```
        print("Level-order display:")
        queue = [self.root]

        while queue:
            current = queue.pop(0)
            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print()

    def delete(self, value):
        if self.root is None:
            print("Tree is empty.")
            return

        if self.root.data == value:
            if self.root.left is None and self.root.right is None:
                self.root = None
                print("Element deleted successfully.")
            else:
                print("Delete its children first!")
            return

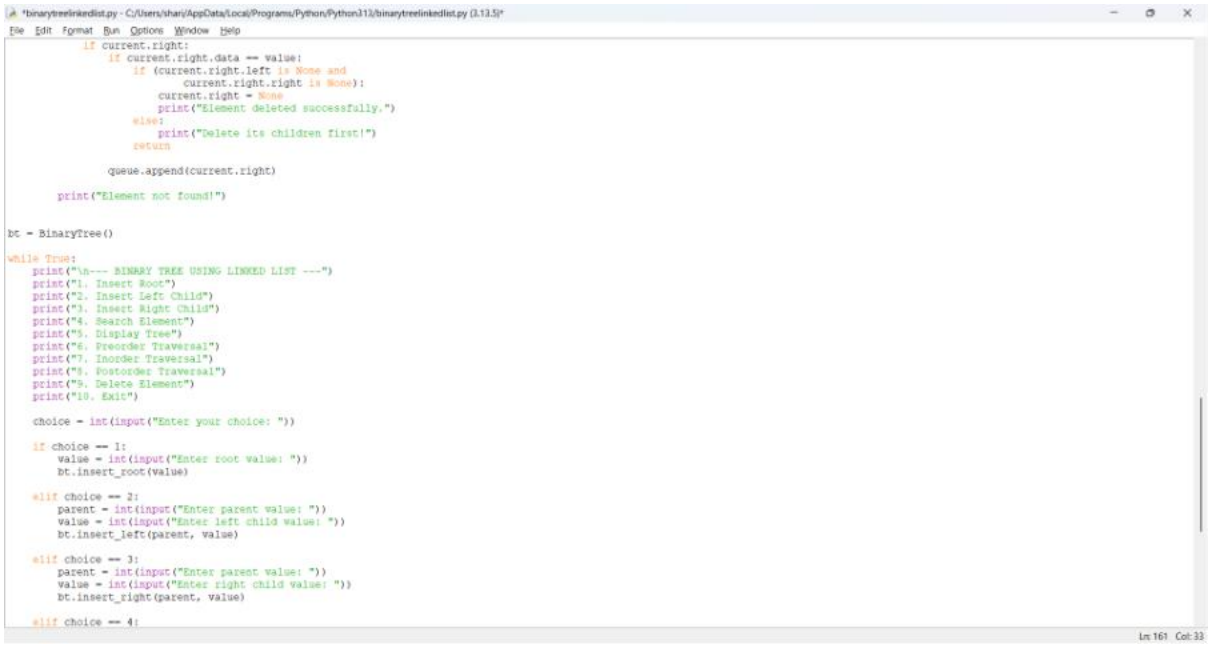
        queue = [self.root]

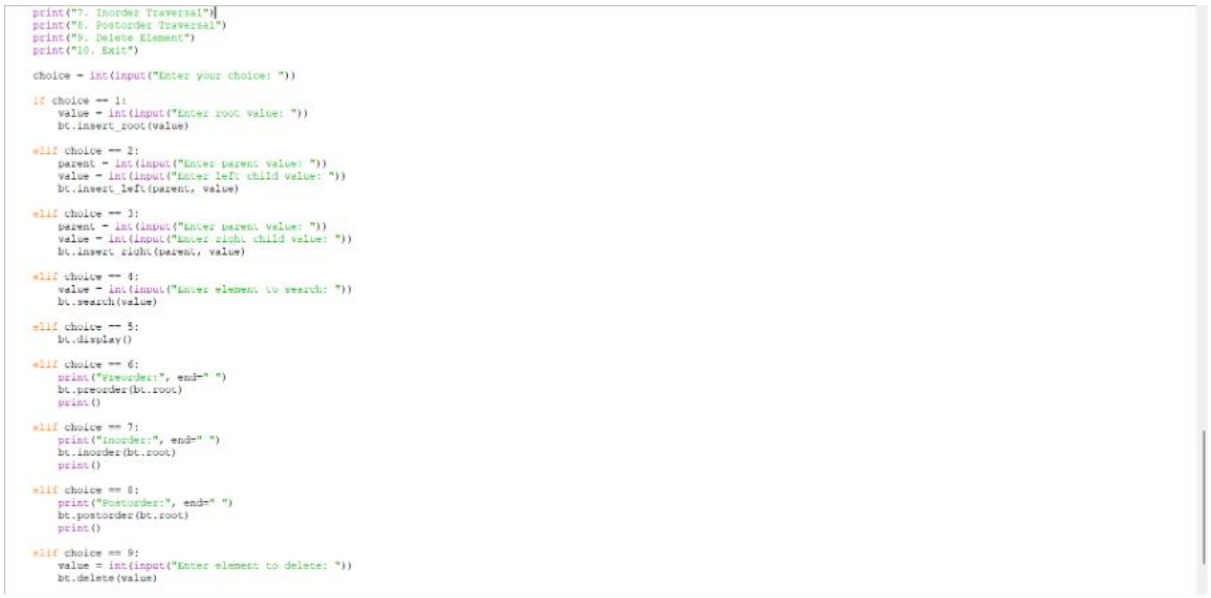
        while queue:
            current = queue.pop(0)

            if current.left:
                if current.left.data == value:
                    if (current.left.left is None and
                            current.left.right is None):
                        current.left = None
                        print("Element deleted successfully.")
                    else:
                        print("Delete its children first!")
                    return

                queue.append(current.left)

            if current.right:
                if current.right.data == value:
```





Output:

```
============================================================ RESTART: C:\Users\BHUVANESH\OneDrive\Desktop\DSA25025\lab7b.py ==================

1. Insert Root
2. Insert Child
3. Search
4. Display
5. Delete Leaf Node
6. Preorder
7. Inorder
8. Postorder
9. Exit
Enter choice: 1
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 2
Enter parent value: 12
Enter left child value: 12
Parent not found!
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 3
Enter parent value: 10
Enter right child value: 3
Right child inserted successfully.
```

```
--- BINARY TREE USING LINKED LIST ---
1. Insert Root
2. Insert Left Child
3. Insert Right Child
4. Search Element
5. Display Tree
6. Preorder Traversal
7. Inorder Traversal
8. Postorder Traversal
9. Delete Element
10. Exit
Enter your choice: 4
Enter element to search: 13
Element not found.
```

Explanation:

1. A binary tree is a non-linear data structure in which each node can have at most two children: a left child and a right child.

2. In linked-list representation, each node is created dynamically using a class.

3. Each node consists of three components: data, left, and right.

4. The data field stores the value of the node, while left and right store references to its child nodes.

5. The root variable points to the first node of the binary tree. If the tree is empty, the root is set to None.

6. Insertion: A new node is created and connected as the left or right child of the specified parent node.

7. Searching: The program searches for a given element using level-order traversal with a queue.

8. Display: The elements of the binary tree are displayed in level order, starting from the root node.

9. Deletion: A leaf node is deleted by setting the corresponding reference of its parent to None.

10. Preorder traversal: Visits the root node, left subtree, and right subtree.

11. Inorder traversal: Visits the left subtree, root node, and right subtree.

12. Postorder traversal: Visits the left subtree, right subtree, and root node.

13. Linked-list representation allocates memory dynamically and does not require a fixed-size array.

14. It is suitable for trees of different shapes, but each node requires additional memory to store references.

Error table:

sno	Error obtained	Error rectification
1	Searching for an element that does not exist in the tree.	Added conditions to display an appropriate message when the element is not found
2	Incorrect traversal of tree elements.	Implemented inorder, preorder, and postorder traversal functions using recursion.

WEEK 8

Aim: write a python program to implement binary search tree and all its operations by using the arrays

Code:

```
tree = []

def insert(value):
    global tree
    if len(tree) == 0:
        tree.append(value)
        print("Inserted successfully")
        return

    i = 0
    while i < len(tree):
        if tree[i] == -1:
            tree[i] = value
            print("Inserted successfully")
            return
        elif value < tree[i]:
            i = 2 * i + 1
        elif value > tree[i]:
            i = 2 * i + 2
        else:
            print("Duplicate value not allowed")
            return

        while i >= len(tree):
            tree.append(-1)

    tree.append(value)


def search(value):
    i = 0
    while i < len(tree) and tree[i] != -1:
        if tree[i] == value:
            return True
        elif value < tree[i]:
            i = 2 * i + 1
        else:
            i = 2 * i + 2

    return False


def inorder(i):
    if i < len(tree) and tree[i] != -1:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)
```

```
def inorder(i):
    if i < len(tree) and tree[i] != -1:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


def preorder(i):
    if i < len(tree) and tree[i] != -1:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def postorder(i):
    if i < len(tree) and tree[i] != -1:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")


while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Inorder")
    print("4. Preorder")
    print("5. Postorder")
    print("6. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif ch == 2:
        value = int(input("Enter value to search: "))
        if search(value):
            print("Value found")
        else:
            print("Value not found")

    elif ch == 3:
        print("Inorder:")
        inorder(0)
```

```
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def postorder(i):
    if i < len(tree) and tree[i] != -1:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")


while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Inorder")
    print("4. Preorder")
    print("5. Postorder")
    print("6. Exit")

    ch = int(input("Enter your choice: "))

    if ch == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif ch == 2:
        value = int(input("Enter value to search: "))
        if search(value):
            print("Value found")
        else:
            print("Value not found")

    elif ch == 3:
        print("Inorder:")
        inorder(0)

    elif ch == 4:
        print("Preorder:")
        preorder(0)

    elif ch == 5:
        print("Postorder:")
        postorder(0)

    elif ch == 6:
        print("Program terminated")
        break
```

OUTPUT:

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
======== RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab8.py ========


1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter your choice: 1
Enter value: 2
Inserted successfully


1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter your choice: 2
Enter value to search: 2
Value found


1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Exit
Enter your choice: 6
Program terminated
>>>
```

Explanation:

1. **Initialization:** An empty list named tree is created to store the elements of the Binary Search Tree (BST).
2. **Insertion Operation:** The insert() function inserts a new element into the tree. If the value is smaller than the current node, it moves to the left child. If the value is greater, it moves to the right child. Duplicate elements are not allowed.
3. **Array Representation:** The tree is represented using an array. For a node at index i, the left child is located at 2*i + 1, and the right child is located at 2*i + 2.

4. **Search Operation:** The search() function searches for a given element. It compares the value with the current node and moves to the left or right child until the element is found or the search fails.
5. **Deletion Operation:** The delete() function removes the specified element from the tree. It collects the remaining elements and rebuilds the BST without the deleted element.
6. **Inorder Traversal:** The inorder() function visits the left subtree, root node, and right subtree. It displays the elements in ascending order.
7. **Preorder Traversal:** The preorder() function visits the root node first, followed by the left subtree and the right subtree.
8. **Postorder Traversal:** The postorder() function visits the left subtree, the right subtree, and finally the root node.
9. **Display Operation:** The display() function displays the array representation of the BST. If the tree is empty, it displays an appropriate message.
10. **Menu-Driven Interface:** The program displays a menu containing all the available operations. The user selects an operation by entering the corresponding choice.
11. **Exit Operation:** When the user selects option 8, the program displays a termination message and stops executing.
12. **Invalid Choice Handling:** If the user enters a choice that is not available in the menu, the program displays an invalid-choice message and allows the user to try again.
13. **Program Termination:** The while True loop repeatedly displays the menu and performs the selected operation until the user chooses to exit.

Top of Form

Bottom of Form

error table:

Sno	Error obtained	Error rectification
1	Incorrect array indexing while inserting elements.	Used the formulas 2*i + 1 for the left child and 2*i + 2 for the right child.
2	Duplicate elements may be inserted into the tree.	Added a condition to check for duplicate elements before insertion.

Week - 9

AIM:write a python program to implement the AVL tree using arrays

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


class AVL:
    def height(self, root):
        if root is None:
            return 0
        return root.height

    def balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)

    def rightRotate(self, y):
        x = y.left
        t = x.right

        x.right = y
        y.left = t

        y.height = 1 + max(self.height(y.left), self.height(y.right))
        x.height = 1 + max(self.height(x.left), self.height(x.right))

        return x

    def leftRotate(self, x):
        y = x.right
        t = y.left

        y.left = x
        x.right = t

        x.height = 1 + max(self.height(x.left), self.height(x.right))
        y.height = 1 + max(self.height(y.left), self.height(y.right))

        return y

    def insert(self, root, key):
        if root is None:
            return Node(key)
```

```
        y.height = 1 + max(self.height(y.left), self.height(y.right))

        return y

    def insert(self, root, key):
        if root is None:
            return Node(key)

        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            print("Duplicate values are not allowed")
            return root

        root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.balance(root)

        # LL Case
        if b > 1 and key < root.left.key:
            return self.rightRotate(root)

        # RR Case
        if b < -1 and key > root.right.key:
            return self.leftRotate(root)

        # LR Case
        if b > 1 and key > root.left.key:
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)

        # RL Case
        if b < -1 and key < root.right.key:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

        return root

    def minValue(self, root):
        while root.left is not None:
            root = root.left
        return root

    def delete(self, root, key):
        if root is None:
            return root
```

```
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.minValue(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.balance(root)

        # LL Case
        if b > 1 and self.balance(root.left) >= 0:
            return self.rightRotate(root)

        # LR Case
        if b > 1 and self.balance(root.left) < 0:
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)

        # RR Case
        if b < -1 and self.balance(root.right) <= 0:
            return self.leftRotate(root)

        # RL Case
        if b < -1 and self.balance(root.right) > 0:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)

        return root

    def search(self, root, key):
        if root is None:
            return False

        if root.key == key:
```

```
            return False

        if root.key == key:
            return True
        elif key < root.key:
            return self.search(root.left, key)
        else:
            return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

    def display(self, root, space=0):
        if root is not None:
            self.display(root.right, space + 5)
            print(" " * space + str(root.key))
            self.display(root.left, space + 5)


# Main Menu
tree = AVL()
root = None

while True:
    print("\n\n--- AVL TREE MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Display")
    print("8. Exit")
```

```
print("8. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    key = int(input("Enter value to insert: "))
    root = tree.insert(root, key)
    print("Node inserted successfully")

elif choice == 2:
    key = int(input("Enter value to delete: "))
    if tree.search(root, key):
        root = tree.delete(root, key)
        print("Node deleted successfully")
    else:
        print("Value not found")

elif choice == 3:
    key = int(input("Enter value to search: "))
    if tree.search(root, key):
        print("Element found")
    else:
        print("Element not found")

elif choice == 4:
    print("Inorder Traversal:")
    tree.inorder(root)
    print()

elif choice == 5:
    print("Preorder Traversal:")
    tree.preorder(root)
    print()

elif choice == 6:
    print("Postorder Traversal:")
    tree.postorder(root)
    print()

elif choice == 7:
    print("AVL Tree:")
    tree.display(root)

elif choice == 8:
    print("Exiting Program...")
    break

else:
```

```
========== RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab9a.py ==========

===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Display
8. Exit
Enter your choice: 1
Enter value: 10
Element inserted successfully
```

Delete element

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 2
Enter value to delete: 20
Node deleted successfully.
```

Search element

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 3
Enter value to search: 60
Value not found.
```

Inorder traversal

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 4
Inorder: 10 30 40
```

Preorder traversal

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 5
Preorder: 30 10 40
```

Postorder traversal

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 6
Postorder: 10 40 30
```

Display tree

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 7
    40
30
    10
```

exit

```
===== AVL TREE USING ARRAYS =====
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 8
Exiting program.
```

Explanation:

- The program implements an AVL tree using arrays in Python.
- The keys, left, right, and height arrays store node values, child indices, and heights.
- The create_node() function creates a new node and initializes its values.
- The get_height() function returns the height of a node.
- The update_height() function updates the height of a node.

- The balance() function calculates the balance factor of a node.
- The right_rotate() function performs a right rotation to balance the tree.
- The left_rotate() function performs a left rotation to balance the tree.
- The insert_node() function inserts new values and balances the tree when necessary.
- The program handles four imbalance cases: LL, RR, LR, and RL.
- The search() function checks whether a value exists in the tree.
- The min_node() function finds the minimum value in a subtree.
- The delete_node() function deletes a node and restores the tree's balance.
- The inorder() function displays values in ascending order.
- The preorder() function visits the root before its children.
- The postorder() function visits the root after its children.
- The display() function displays the tree structure.
- The main program provides a menu for insertion, deletion, searching, traversals, and displaying the tree.
- The try-except block handles invalid integer inputs.
- The program maintains the AVL tree's balance to make searching, insertion, and deletion efficient.

Error table:

sno	Error obtained	Error rectification
1	Incorrect balance factor	Calculated the balance factor as left subtree height minus right subtree height.
2	Root index not updated after rotation	Stored the returned root index in avl.root.

Week-10

Aim: write a python program to hashing

Code:

```
SIZE = 10

# Hash Table
chaining = [[] for i in range(SIZE)]
linear = [None] * SIZE
quadratic = [None] * SIZE

# Hash Function
def hash_function(key):
    return key % SIZE

# Separate Chaining
def chain_insert(key):
    pos = hash_function(key)
    if key not in chaining[pos]:
        chaining[pos].append(key)
        print("Element inserted successfully")
    else:
        print("Element already exists")

def chain_search(key):
    pos = hash_function(key)
    if key in chaining[pos]:
        print("Element found at index", pos)
    else:
        print("Element not found")

def chain_display():
    print("Hash Table using Separate Chaining:")
    for i in range(SIZE):
        print(i, ":", chaining[i])


# Linear Probing
def linear_insert(key):
    pos = hash_function(key)

    for i in range(SIZE):
        index = (pos + i) % SIZE
        if linear[index] is None:
            linear[index] = key
            print("Element inserted at index", index)
            return
        if linear[index] == key:
            print("Element already exists")
            return
```

```
def linear_insert(key):
    pos = hash_function(key)

    for i in range(SIZE):
        index = (pos + i) % SIZE
        if linear[index] is None:
            linear[index] = key
            print("Element inserted at index", index)
            return
        if linear[index] == key:
            print("Element already exists")
            return

    print("Hash Table is Full")

def linear_search(key):
    pos = hash_function(key)

    for i in range(SIZE):
        index = (pos + i) % SIZE

        if linear[index] is None:
            break
        if linear[index] == key:
            print("Element found at index", index)
            return

    print("Element not found")

def linear_display():
    print("Hash Table using Linear Probing:")
    for i in range(SIZE):
        print(i, ":", linear[i])


# Quadratic Probing
def quadratic_insert(key):
    pos = hash_function(key)

    for i in range(SIZE):
        index = (pos + i * i) % SIZE

        if quadratic[index] is None:
            quadratic[index] = key
            print("Element inserted at index", index)
            return
        if quadratic[index] == key:
            print("Element already exists")
```

```
        if quadratic[index] is None:
            quadratic[index] = key
            print("Element inserted at index", index)
            return
        if quadratic[index] == key:
            print("Element already exists")
            return

    print("Hash Table is Full")

def quadratic_search(key):
    pos = hash_function(key)

    for i in range(SIZE):
        index = (pos + i * i) % SIZE

        if quadratic[index] is None:
            break
        if quadratic[index] == key:
            print("Element found at index", index)
            return

    print("Element not found")

def quadratic_display():
    print("Hash Table using Quadratic Probing:")
    for i in range(SIZE):
        print(i, ":", quadratic[i])


# Main Menu
while True:
    print("\n===== HASHING TECHNIQUES =====")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 4:
        print("Program terminated.")
        break

    if choice not in [1, 2, 3]:
        print("Invalid choice")
        continue
```

```
    if choice == 4:
        print("Program terminated.")
        break

    if choice not in [1, 2, 3]:
        print("Invalid choice")
        continue

    while True:
        print("\n1. Insert")
        print("2. Search")
        print("3. Display")
        print("4. Back to Main Menu")

        operation = int(input("Enter your operation: "))

        if operation == 4:
            break

        if operation == 1:
            key = int(input("Enter element to insert: "))

            if choice == 1:
                chain_insert(key)
            elif choice == 2:
                linear_insert(key)
            else:
                quadratic_insert(key)

        elif operation == 2:
            key = int(input("Enter element to search: "))

            if choice == 1:
                chain_search(key)
            elif choice == 2:
                linear_search(key)
            else:
                quadratic_search(key)

        elif operation == 3:
            if choice == 1:
                chain_display()
            elif choice == 2:
                linear_display()
            else:
                quadratic_display()

        else:
```

Output:

Insert element

```
= RESTART: C:/Users/BHUVANESH/OneDrive/Desktop/DSA25025/lab10.py

===== HASHING TECHNIQUES =====
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 1

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 1
Enter element to insert: 10
Element inserted successfully

1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation:
```

Search element

```
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 2
Enter the element: 34
Element not found.
```

display

```
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 3

Hash Table using Separate Chaining:
0 : []
1 : [12, 23]
2 : []
3 : []
4 : []
5 : []
6 : []
7 : []
8 : []
9 : []
10 : [54]
```

Back to menu

```
1. Insert
2. Search
3. Display
4. Back to Main Menu
Enter your operation: 4

===== HASHING TECHNIQUES =====
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
4. Exit
Enter your choice: 2
```

Explanation:

- The program implements three hashing techniques: Separate Chaining, Linear Probing, and Quadratic Probing.
- The SIZE = 11 statement defines the size of the hash table as 11.
- The chaining list stores elements using separate chaining.
- The linear list stores elements using linear probing.
- The quadratic list stores elements using quadratic probing.
- The insert_chaining() function inserts an element into the corresponding index using the modulo operation.
- The search_chaining() function searches for an element in the corresponding chain.
- The display_chaining() function displays all the elements stored in the separate chaining hash table.

- The insert_linear() function inserts an element by checking consecutive positions when a collision occurs.
- The search_linear() function searches for an element using linear probing.
- The display_linear() function displays the elements in the linear probing hash table.
- The insert_quadratic() function resolves collisions by checking positions using the quadratic formula.
- The search_quadratic() function searches for an element using quadratic probing.
- The display_quadratic() function displays the elements in the quadratic probing hash table.
- The modulo operation key % SIZE calculates the initial hash index.
- The None value indicates an empty position in the linear and quadratic probing tables.
- The main menu allows the user to select a hashing technique or exit the program.
- The submenu allows the user to insert, search, display elements, or return to the main menu.
- The if-elif-else statements execute the selected operation.
- The program demonstrates how collisions are handled using three different hashing techniques.

Error table:

sno	Error obtained	Error rectification
1	Collision occurs when two elements have the same hash index.	Used separate chaining, linear probing, and quadratic probing to handle collisions.
2	Incorrect hash index calculation.	Used the modulo operation key % SIZE to calculate the hash index.